

과B근d 부트 간리부트 i 상범도능버드범사방 뒤 뒤무능사범분 땀 가상범범범사범론 계뜨범범부g 능능
s 능능론버드범사범 드범사범범범 능능테분분b d f 능능가상도성범v 범범범범가 r

DI 잠 고m~t 감 2d~t 같범범 W에Pt 감 계같 h t 감 .a개범개 f 감략 감범개
5 DTMO 868: .T000: 6것 감범 감범 것범범 각범범y 감범y 같범nt 것 0감 잠y 감범 /
D 7:
D.O/

a같범개 감dbW		4개 적v것범		NL.Pt 감범	
4		2		2	
=		2Y gMM/2		.	
2		4TMO 868:		2	
0개범개 감 적v것범		2		7Æ66	
4				4	

DB?C T 7 b
D
% 1 %
D
74l y감
NL NL
개 적v것범
84Z 같
2
a같범개 감dbW
94l y감 감 Z같
. 2Y gMM
. 2 /
D9 4% 2 %

DB?D M

CT 3e 상범론부 범범범

a같범개 감dbW D

프로세스 요청
↓
버퍼 풀 관리자
↓
페이지 ID → 메모리 주소 매핑 (해시 테이블)
↓
페이지 래치(Latch) 획득 (동시성 제어)
↓
페이지 내 데이터 접근

.SZdh / D

HNSW 탐색 (상위 층에서 하위 층으로):

1. 현재 노드 X에서 가까운 이웃 Y 찾기
2. 이웃 Y를 다음 노드로 이동
3. Y에서 다시 이웃 Z 찾기
4. 이 과정 반복 (수십~수백 번의 노드 방문)

D .W갯~/

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)
- 페이지 읽기 후: 래치 해제

결과:

래치 획득/해제 시간	
→ 전체 검색 시간의 30~50%	
비교: Faiss (메모리 직접 접근)	
→ 래치 오버헤드 0%	

D

100만 벡터, k=10 검색:

PostgreSQL pgvector: 150ms (래치 오버헤드 포함)

Faiss (메모리): 8ms (래치 오버헤드 없음)

비율: $150/8 = 18.75$ 배 느림 (거의 전부가 래치 오버헤드)

|

I Od4 a갯갯갯dbW 4

2Y gMM2

a갯갯갯dbW

NL

DT 3갯갯갯dbW

a갯갯갯dbW D

1단계: 페이지 버퍼 풀에서 찾기

- └ 페이지 ID 계산 (block number)
- └ 해시 테이블 조회
- └ 버퍼 캐시에서 페이지 위치 확보

2단계: 페이지 내 튜플 찾기

- └ 페이지 헤더 해석 (특수 공간 할당, 튜플 오프셋 배열)
- └ 튜플 오프셋 계산
- └ 해당 위치로 포인터 이동

3단계: 튜플 헤더 파싱

- └ 트랜잭션 ID (xmin, xmax)
- └ 커맨드 ID (cmin, cmax)
- └ 가시성 정보 (deleted?)
- └ 락 정보

4단계: MVCC 가시성 확인

- └ 현재 트랜잭션에서 보여야 하는 튜플인가?
- └ 스냅샷 확인
- └ 트랜잭션 히스토리 조회 가능

5단계: 실제 데이터 추출

- └ 컬럼 오프셋 계산
- └ 벡터 데이터 추출

D

백만 벡터를 가진 인덱스에서 top-10 검색:

각 후보 벡터마다:

1. 거리 계산: $1,536\text{차원} \times 4\text{바이트} = 6,144\text{ 바이트 메모리 접근}$
2. 튜플 헤더 파싱: $\sim 200\text{ 바이트 메모리 접근}$
3. MVCC 확인: 메모리 참조 5~10회

오버헤드: 거리 계산 대비 튜플 오버헤드가 30~50%

DNZK .: 6C?N/

Faiss (메모리 기반):

거리 계산: $4KB \times N$

추가 오버헤드: 거의 없음

총 시간: 순수 거리 계산 시간

PostgreSQL:

거리 계산: $4KB \times N$

튜플 헤더 파싱: 200B x N

MVCC 확인: 10참조 × N

총 시간: 거리 계산 + 오버헤드 (이것이 20~50% 추가)

1

1 Od4Y gMM a갸갸갸dbW 4

Y gMM

.gLKd0 /

×간간거갸u궤.

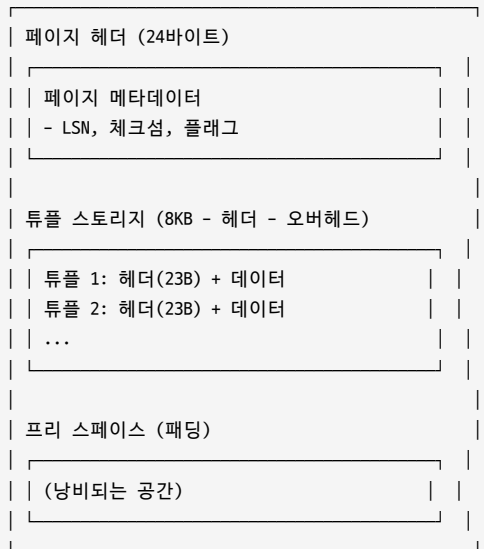
/

Y gMM

FT 3기 벤딩 d 버 백발 타는 사람 만

$$a_{\text{검출기}} \geq a_{\text{검출기}} + \delta a_{\text{검출기}} \quad D$$

8KB 페이지:



SZdh D

74

SZdh	D	1
------	---	---

D766 리 BVL 리 A2C66

84Y gMM

경간+경간+경간+경간+경간+경간+경간+경간
766 도8: G 8: Y L

94

D

원본 벡터 데이터: 100MB (100만 벡터, 1KB 각)

HNSW 인덱스:

Faiss: ~150MB (벡터 + 그래프)

PostgreSQL: ~1,200MB (벡터 + 그래프 + 페이지 오버헤드 + MVCC)

비율: 1,200 / 150 = 8배 (공간 증폭)

결과:

- 인덱스가 메모리에 못 들어감

- 디스크 I/O 발생

- 검색 성능 100배 악화

D

스토리지 계층	용량	접근시간
L1 캐시	32KB	0.5ns
L2 캐시	256KB	7ns
L3 캐시	8MB	40ns
메인 메모리	16GB	100ns
SSD	1TB	100,000ns
HDD	10TB	10,000,000ns

Faiss (150MB):

대부분이 메인 메모리에 올라감

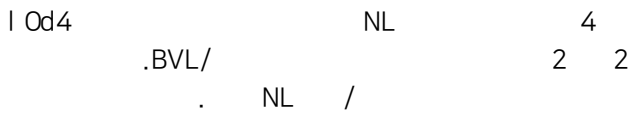
평균 접근 시간: ~100ns

PostgreSQL (1,200MB):

일부가 SSD로 스푼

평균 접근 시간: ~10,000ns (100배 느림)

I



LT

a갑갓갓dbW SZdhD

1. 벡터 데이터를 삽입 (INSERT)

└ 버퍼 풀 경유 (래치 획득)

└ MVCC 메타데이터 작성

└ WAL(Write-Ahead Log) 기록

2. 각 벡터마다 HNSW 그래프 갱신

└ 기존 노드들과 거리 계산

└ 거리 기반 이웃 선택

└ 그래프 연결 정보 갱신

└ 다시 WAL 기록

3. 메모리 부족 시

└ maintenance_work_mem 초과 → 디스크 스푼

→ 스푼된 데이터 다시 읽기

→ 다시 버퍼 풀 경유

결과:

벡터 1개 삽입 = 20~50회 페이지 I/O

D

데이터셋: 1,000만 벡터 (1536D, 약 60GB)

PostgreSQL pgvector:

구축 시간: 48시간
처리 속도: ~57벡터/초

Faiss:

구축 시간: 30분
처리 속도: ~5,500벡터/초

비율: 5,500 / 57 = 96배 빠름

D

Faiss:

- 메모리에서 직접 작업
- 거리 계산 + 그래프 갱신만 필요
- I/O 없음 (최종 저장 시에만)

PostgreSQL:

- 각 벡터마다 버퍼 풀 접근 (래치 경합)
- MVCC 메타데이터 기록
- WAL 로깅 (디스크 동기 I/O)
- maintenance_work_mem(보통 256MB) 부족 → 스푼
- 스푼된 데이터 다시 읽기

I

$I_{\text{Od4h}} \text{ KW} / a_{\text{검색}} \text{ dbW} = 4$

$NL \quad h \text{ KW}$

$D \quad h \text{ KW} \quad / \quad . \quad /$

$MT \quad 정$

$a_{\text{검색}} \text{ dbW} \quad D$

일반적인 필터:

WHERE age > 30
추정: 히스토그램 사용 → 정확도 ~90%

벡터 필터:

WHERE embedding <-> query_vec < 0.5
추정: "알 수 없음" → 기본값 사용

pgvector의 기본값:

선택도 = 33.3% (HNSW는 모든 쿼리에서 1/3을 반환한다고 가정)

D

벤치마크: Wikipedia 이미지 임베딩 (1,000만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100개 찾기)
실제 선택도: 0.001% (100 / 10,000,000)
pgvector 추정: 33.3%
오류: 33,300배 과대 추정

결과: 옵티마이저가 극도로 비효율적인 조인 순서 선택

D

쿼리:

```
SELECT o.order_id
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
WHERE l.product_embedding <-> query_vec < 0.5
```

pgvector의 추정 (잘못됨):

orders: 100만 행 → Seq Scan
lineitem: 600만 행 × 33.3% = 200만 행 필터링
계획: Seq Scan orders → 600만 행 각각 lineitem 스캔

최적 계획:
lineitem 먼저 필터링 (백터) → 100행
그 다음 orders 조인 → 100행 결과

D

pgvector (33.3% 추정):
쿼리 시간: 850초

VBASE (50% 추정):
쿼리 시간: 85초

정확한 추정 (0.001%):
쿼리 시간: 2초

비율: 850초 / 2초 = 425배 느림

I

I Od4 4
. 2 /

D
.O경개일면 /D
. /D
.OMb` /D

DBF T 7 b

D

과33333과333333333과3333333과		
		.gLKd0/
		간간개일면u와 .Y gMM /
		.리 /
		h KW . / 1
		.O경개일면

D 2Y gMM2 4 4

DBL

= D

O경개일면 D

한계 5: 고정 선택도 (33.3%)
→ Exqutor의 ECQ0: 정확한 카디널리티 추정
→ 결과: pgvector에서 1,000배 성능 향상

D

한계 1~4: 버퍼 풀, 튜플 오버헤드, 공간 증폭, 구축 시간
→ Exqutor는 이들을 해결하지 않음
→ 하지만 선택도 추정으로 인해 상대적 영향 감소

D

pgvector 기본:
비효율적 계획 선택으로 인한 성능 저하: 1,000배
+ 버퍼 풀 오버헤드: 별도로 ~20배
종합 오버헤드: 1,000배 × 20배 = 20,000배 이상

정확한 계획 선택으로 오버헤드 제거: 1,000배 → 1배
버퍼 풀 오버헤드 여전함: ~20배
종합 오버헤드: 1배 × 20배 = 20배

1,000배 → 20배: 실질 향상 50배 ($1,000/20$)

개 적v결합64명 4 D
개 적v결합64 교64AD
SZdh . /

D

999+ /

경계면

74 Pt $\times$ 2Pt NL 2Pt D
Pt $\times$ 2Pt D
D
D
KMN D
Wea D

개 격	v	결 격	Pt	격	%	NL	격	%	2%	NL	격	NL %	4
-----	---	-----	----	---	---	----	---	---	----	----	---	------	---

84 NL

a갈v관 dbW D

Y개bW I

dbW았 I

`관v관 I

NL 2

94

D
gLKdO/
/
/
.

4

3 4

DB?	P	T
1	0.000000	0.000000
2	0.000000	0.000000
3	0.000000	0.000000
4	0.000000	0.000000
5	0.000000	0.000000
6	0.000000	0.000000
7	0.000000	0.000000
8	0.000000	0.000000
9	0.000000	0.000000
10	0.000000	0.000000
11	0.000000	0.000000
12	0.000000	0.000000
13	0.000000	0.000000
14	0.000000	0.000000
15	0.000000	0.000000
16	0.000000	0.000000
17	0.000000	0.000000
18	0.000000	0.000000
19	0.000000	0.000000
20	0.000000	0.000000
21	0.000000	0.000000
22	0.000000	0.000000
23	0.000000	0.000000
24	0.000000	0.000000
25	0.000000	0.000000
26	0.000000	0.000000
27	0.000000	0.000000
28	0.000000	0.000000
29	0.000000	0.000000
30	0.000000	0.000000
31	0.000000	0.000000
32	0.000000	0.000000
33	0.000000	0.000000
34	0.000000	0.000000
35	0.000000	0.000000
36	0.000000	0.000000
37	0.000000	0.000000
38	0.000000	0.000000
39	0.000000	0.000000
40	0.000000	0.000000
41	0.000000	0.000000
42	0.000000	0.000000
43	0.000000	0.000000
44	0.000000	0.000000
45	0.000000	0.000000
46	0.000000	0.000000
47	0.000000	0.000000
48	0.000000	0.000000
49	0.000000	0.000000
50	0.000000	0.000000
51	0.000000	0.000000
52	0.000000	0.000000
53	0.000000	0.000000
54	0.000000	0.000000
55	0.000000	0.000000
56	0.000000	0.000000
57	0.000000	0.000000
58	0.000000	0.000000
59	0.000000	0.000000
60	0.000000	0.000000
61	0.000000	0.000000
62	0.000000	0.000000
63	0.000000	0.000000
64	0.000000	0.000000
65	0.000000	0.000000
66	0.000000	0.000000
67	0.000000	0.000000
68	0.000000	0.000000
69	0.000000	0.000000
70	0.000000	0.000000
71	0.000000	0.000000
72	0.000000	0.000000
73	0.000000	0.000000
74	0.000000	0.000000
75	0.000000	0.000000
76	0.000000	0.000000
77	0.000000	0.000000
78	0.000000	0.000000
79	0.000000	0.000000
80	0.000000	0.000000
81	0.000000	0.000000
82	0.000000	0.000000
83	0.000000	0.000000
84	0.000000	0.000000
85	0.000000	0.000000
86	0.000000	0.000000
87	0.000000	0.000000
88	0.000000	0.000000
89	0.000000	0.000000
90	0.000000	0.000000
91	0.000000	0.000000
92	0.000000	0.000000
93	0.000000	0.000000
94	0.000000	0.000000
95	0.000000	

D

74

개 격v것번aKdO M
2Y gMMh KW

84
s

D

D7=6간검

D7: 6간검 / 리 76간검

Y gMM D786간검 / 리 86간검

h KW D766간검 / 리 86간검

DB간검Pt 검 / 리 C8간검

s

94

a검v것번dbW . 2 /

dTPe378BY D74B .78BN/

Q검v것번D77C .966N/

Y d M` M` D789 .86: BN/

D 5

응답 시간 = f(벡터 크기, 벡터 차원, 쿼리 유형)

기본 메트릭:

- 단일 쿼리 지연시간 (ms)
- 처리량 (벡터/초)
- 메모리 사용량 (GB)
- 저장소 공간 (GB)

DB?

D

74 NL NL

개 격v것번 %dbW %

84

.

/

O강v것번 개 격v것번

94

D NL .Y 검v것번h yt 검v것번/ gLKdO
Da검v것번dbW1 개 격v것번 O강v것번

C?

개 격v것번 2% I % D
a검v것번dbW
3
%개 격v것번 %

D?

D

```
SELECT * FROM vectors
WHERE embedding <-> query_vec < threshold
```

D

```
SELECT v.id, m.metadata, COUNT(*) AS cnt
FROM vectors v
JOIN metadata m ON v.id = m.vector_id
WHERE v.embedding <-> query_vec < threshold
      AND m.created_at > NOW() - INTERVAL '7 days'
GROUP BY v.id, m.metadata
ORDER BY cnt DESC
```

개 적v것언4

F?

2 .것vt 각D

SZdh

개 적v것언Pt 각vt 각

